

RAPPORT DE PROJET

Mini-Projet PPSE

Réalisé par :
Stéphane JI
Rabab BOUDIH

Année Universitaire 2025-2026

Sommaire

Résultats et analyses 3
Axe II - Focusing of speeding up one block 3
Task II-1, 2 & 3 [difficulty: easy to medium] 3
1. L'impact des paramètres de quantification sur le BER/FER 3
2. L'efficacité de l'architecture NEON selon la taille des trames (K) 3
Task II-4. Bit-packing [difficulty: medium] 5
1. Résultats en virgule flottante (Hard et Soft) 6
2. Résultats en virgule fixe (Soft FP et Hard FP) 7
Axe I - Speeding up the overall simulation throughput 9
Task I-1. Multi-threading [difficulty: medium] 9
Les résultats : 9
1. Thread Standard Vs Thread Packed 9
2. Thread Standard Vs Thread NEON 12
3. Thread Packed Vs Thread Neon 13
Conclusion Générale 14
Configuration et lancement 15
Procédure de compilation 15
Lancer le code séquentiel 15
Lancer le code avec les thread 15
Présentation des scripts d'analyse Python 16
1. Comparaison des décisions Soft 16
2. Comparaison des décisions Hard et Soft 16

Résultats et analyses

Dans le cadre de ce mini-projet, nous avons d'abord poursuivi l'implémentation NEON en traitant l'Axe II (Tâches 1, 2 et 3). Ensuite, nous avons intégré le *bit-packing* (Tâche II-4). Enfin, nous avons implémenter la parallélisation avec les threads (Tâche I-1). De plus, on va définir `Normal` les résultats des simulations du lab3.

Axe II - Focusing of speeding up one block

Task II-1, 2 & 3 [difficulty: easy to medium]

1. *Speedup the modulator.* [difficulty: easy] Vectorize it with NEON instructions.
2. *Speedup the demodulator.* [difficulty: easy] Vectorize it with NEON instructions.
3. *Speedup the monitor.* [difficulty: easy to medium] Vectorize it with NEON instructions.

Pour tester cette partie, nous avons exécuté simultanément tous les blocs NEON en utilisant les mots-clés `rep-hard8-neon` et `rep-soft8-neon`. Cela nous permet de traiter 4 blocs avec des instructions *SIMD*.

1. L'impact des paramètres de quantification sur le BER/FER

Sur les graphiques actuels, les courbes des simulations 2 et 3 présentent un léger décalage. Ce comportement n'est pas dû à une erreur d'implémentation du décodeur NEON, mais à l'utilisation de paramètres de quantification (S et F) inadaptés. Les valeurs utilisées ici étaient celles optimisées pour le Lab 4, mais elles ne conviennent pas aux spécificités de ce mini-projet. En appliquant les valeurs de quantification optimales, les performances de correction d'erreurs (BER et FER) de la version NEON en virgule fixe s'aligneront parfaitement sur celles de la version standard en virgule flottante.

2. L'efficacité de l'architecture NEON selon la taille des trames (K)

Une fois la validité mathématique confirmée (BER/FER identiques), l'analyse des temps d'exécution met en évidence le véritable comportement des instructions vectorielles *SIMD* :

Dans le fichier `/resultats_stats_normaux_lab4/sim1_stats.txt` :

```
================================================================================
Bloc 15               Avg (s)    Min (s)    Max (s)    Avg Mbps   % Temps
--------------------------------------------------------------------------------
Temps Source       : 0.000002, 0.000000, 0.000129,    18.407,   6.06 %
Temps Répétition   : 0.000001, 0.000000, 0.000145,   102.923,   4.33 %
Temps Modulation   : 0.000001, 0.000000, 0.000113,   103.337,   4.32 %
Temps AWGN         : 0.000014, 0.000000, 0.000170,     9.322,  47.85 %
Temps Démodulation : 0.000001, 0.000000, 0.000137,   100.645,   4.43 %
Temps Décodage     : 0.000004, 0.000000, 0.000156,     8.957,  12.45 %
Temps Check        : 0.000001, 0.000000, 0.000128,    40.303,   2.77 %
================================================================================
```

Dans le fichier `/resultats_stats_neon/sim1_stats.txt` :

```
================================================================================
Bloc 15               Avg (s)    Min (s)    Max (s)    Avg Mbps   % Temps
--------------------------------------------------------------------------------
Temps Source       : 0.000002, 0.000000, 0.000081,    18.478,   7.78 %
Temps Répétition   : 0.000001, 0.000000, 0.000124,   107.211,   5.37 %
Temps Modulation   : 0.000001, 0.000000, 0.000065,   176.837,   3.25 %
Temps AWGN         : 0.000011, 0.000000, 0.000421,    11.173,  51.48 %
Temps Démodulation : 0.000001, 0.000000, 0.000062,   194.028,   2.96 %
Temps Décodage     : 0.000001, 0.000000, 0.000093,    48.843,   2.94 %
Temps Check        : 0.000001, 0.000000, 0.000130,    50.138,   2.87 %
================================================================================
```

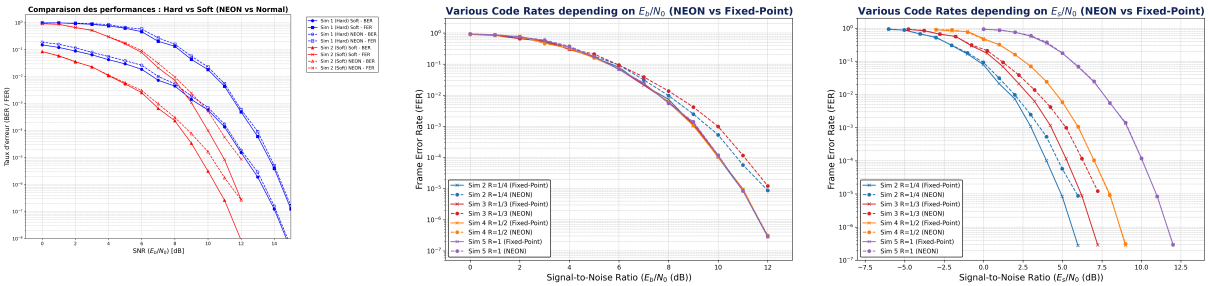


Fig. 1. – Résultats des simulations NEON pour $K = 32$

Pour les trames courtes ($K = 32$) : L'utilisation de NEON réduit drastiquement le temps de calcul brut du décodeur. Toutefois, l'impact sur le débit global de la simulation (en Mbps) reste marginal. Le temps gagné par la vectorisation est en partie masqué par les autres blocs de la chaîne (comme le canal AWGN) et par le coût d'initialisation des registres NEON (overhead), qui est trop lourd par rapport à la petite quantité de données traitées.

Dans le fichier `/resultats_stats_neon/sim1_k256_stats.txt`

```
================================================================================
Bloc 15               Avg (s)    Min (s)    Max (s)    Avg Mbps   % Temps
--------------------------------------------------------------------------------
Temps Source       : 0.000009, 0.000000, 0.000067,    27.254,   8.50 %
Temps Répétition   : 0.000005, 0.000000, 0.000099,   197.485,   4.69 %
Temps Modulation   : 0.000001, 0.000000, 0.000069,   788.069,   1.18 %
Temps AWGN         : 0.000086, 0.000000, 0.000470,    11.843,  78.26 %
Temps Démodulation : 0.000001, 0.000000, 0.000042,  1230.106,   0.75 %
Temps Décodage     : 0.000001, 0.000000, 0.000058,   342.634,   0.68 %
Temps Check        : 0.000001, 0.000000, 0.000095,   387.632,   0.60 %
================================================================================
```

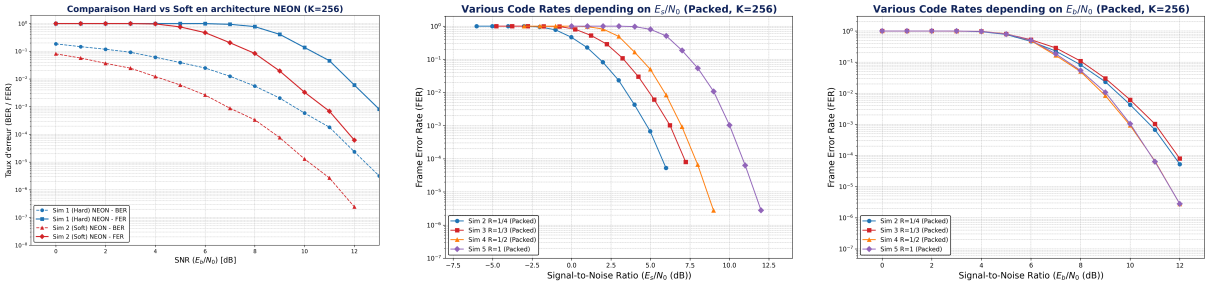


Fig. 2. – Résultats des simulations NEON pour $K = 256$

Pour les trames longues ($K = 256$) : Le débit global (Mbps) connaît une augmentation spectaculaire. Le coût d'initialisation des registres est totalement amorti par le volume de données. Cependant la probabilité d'avoir une erreur est plus élevée comme on peut le voir dans la figure 2 graphe du milieu pour les simulation 4 et 5.

Conclusion

L'intégration de la bibliothèque NEON démontre que la vectorisation n'est pas une solution miracle universelle : elle devient véritablement pertinente et surpuissante uniquement lorsqu'elle est appliquée à des trames de données suffisamment grandes pour saturer efficacement l'architecture matérielle du processeur.

Task II-4. Bit-packing [difficulty: medium]

> *One may have noticed that the source ans the repetition encoder are using 1 byte to store 1 bit. This is not very efficient and the number of load/store could be drastically reduced (e.g. divided by 8). To do this, you can bit-pack/bit-unpack data to load/store bytes that contain 8 bits of information. Be careful, if you do this, you will have to unpack bits also in the modulator.*

Pour cette partie on a utilisé l'instruction : `__builtin_popcount(val);`, c'est une instruction qui permet de dire au compilateur d'optimiser le code et de faire le calcul sans boucle, l'instruction matérielle est `POPCNT`.

En outre, nous avons intégré de nouvelles simulations (`rep-bit-packing-hard`, `rep-bit-packing-soft`, `rep-bit-packing-hard8`, `rep-bit-packing-soft8`) dédiées à l'évaluation de la technique de ***bit-packing*** couplée aux décodages Hard, Soft, Hard FP (virgule fixe) et Soft FP.

Suite à cette implémentation, la recompilation des anciennes simulations du Lab 3 a démontré que le compilateur optimise déjà massivement le code standard, générant des débits de référence beaucoup plus élevés qu'auparavant. Par conséquent, notre analyse de performance se concentrera spécifiquement sur les gains de débit (Mbps) des blocs directement impactés par nos optimisations : **la Source** et **l'Encodeur à Répétition**.

1. Résultats en virgule flottante (Hard et Soft)

Statistiques de performance :

Dans le fichier `/resultats_stats_normaux/sim1_stats.txt` (Standard) :

```
================================================================================
Bloc 15               Avg (s)    Min (s)    Max (s)    Avg Mbps   % Temps
--------------------------------------------------------------------------------
Temps Source       : 0.000002, 0.000000, 0.000141,    18.541,   7.60 %
Temps Répétition   : 0.000001, 0.000000, 0.000277,   107.543,   5.24 %
Temps Modulation   : 0.000001, 0.000000, 0.000061,   193.806,   2.91 %
Temps AWGN         : 0.000011, 0.000000, 0.000354,    11.247,  50.09 %
Temps Démodulation : 0.000001, 0.000000, 0.000302,   194.112,   2.90 %
Temps Décodage     : 0.000001, 0.000000, 0.000336,    25.351,   5.56 %
Temps Check        : 0.000001, 0.000000, 0.000040,    47.141,   2.99 %
================================================================================
```

Dans le fichier `/resultats_stats_packed/sim1_hard_stats.txt` (Packed) :

```
================================================================================
Bloc 15               Avg (s)    Min (s)    Max (s)    Avg Mbps   % Temps
--------------------------------------------------------------------------------
Temps Source       : 0.000001, 0.000000, 0.000393,    41.995,   3.58 %
Temps Répétition   : 0.000001, 0.000000, 0.000101,   185.218,   3.24 %
Temps Modulation   : 0.000001, 0.000000, 0.000149,   186.389,   3.22 %
Temps AWGN         : 0.000011, 0.000000, 0.000497,    11.183,  53.72 %
Temps Démodulation : 0.000001, 0.000000, 0.000057,   193.928,   3.10 %
Temps Décodage     : 0.000001, 0.000000, 0.000090,    25.336,   5.93 %
Temps Check        : 0.000001, 0.000000, 0.000080,    48.938,   3.07 %
================================================================================
```

Analyse des graphes :

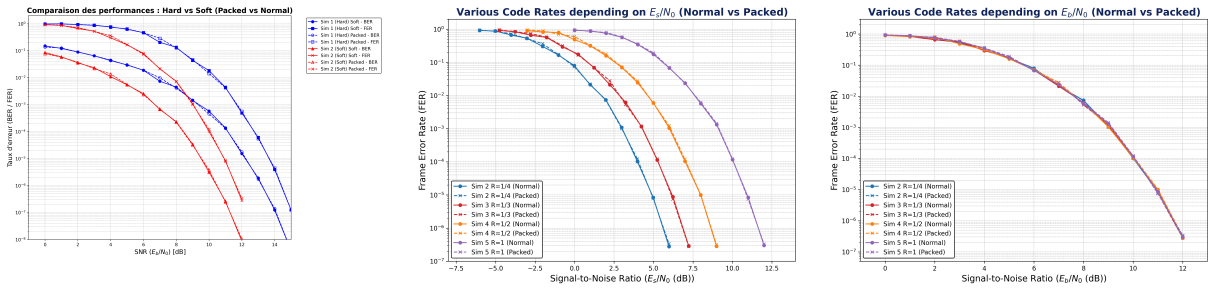


Fig. 3. – Résultats des simulations Packed Flottante $K = 32$

L'implémentation du *bit-packing* sur les trames courtes ($K = 32$) démontre déjà une nette supériorité en termes de débit pour les blocs **Source** et **Répétition** par rapport à la version classique. Concernant les taux d'erreurs, le décalage visible sur les courbes des simulations 2 et 3 ne remet pas en cause l'algorithme, mais découle de l'utilisation de paramètres de quantification (S et F) non optimisés pour cette dynamique spécifique (hérités du Lab 4).

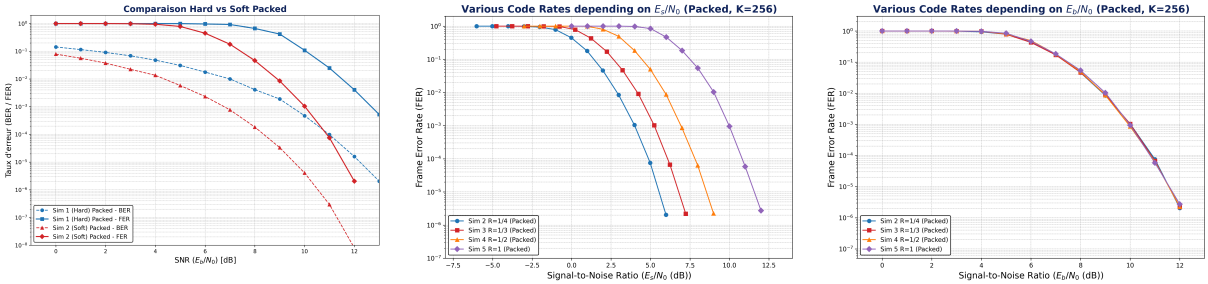


Fig. 4. – Résultats des simulations Packed Flottante $K = 256$

Le passage aux trames longues ($K = 256$) confirme la robustesse de l'implémentation : les courbes de probabilité d'erreur (BER/FER) s'alignent parfaitement avec les attentes théoriques. De plus, l'amortissement du coût d'initialisation sur de grands tableaux rend le débit de la version **packed** significativement plus rapide.

2. Résultats en virgule fixe (Soft FP et Hard FP)

Statistiques de performance (Comparatif $K = 32$ vs $K = 256$) :

Dans le fichier `/resultats_stats_packed/sim1_hard_stats.txt` ($K = 32$) :

```
================================================================================
Bloc 15               Avg (s)    Min (s)    Max (s)    Avg Mbps   % Temps
--------------------------------------------------------------------------------
Temps Source       : 0.000001, 0.000000, 0.000393,    41.995,   3.58 %
Temps Répétition   : 0.000001, 0.000000, 0.000101,   185.218,   3.24 %
...
================================================================================
```

Dans le fichier `/resultats_stats_packed/sim1_hard8_k256_stats.txt` ($K = 256$) :

```
================================================================================
Bloc 15               Avg (s)    Min (s)    Max (s)    Avg Mbps   % Temps
--------------------------------------------------------------------------------
Temps Source       : 0.000002, 0.000000, 0.000083,   145.693,   1.66 %
Temps Répétition   : 0.000001, 0.000000, 0.000031,   863.554,   1.12 %
Temps Modulation   : 0.000002, 0.000000, 0.000036,   654.314,   1.48 %
Temps AWGN         : 0.000088, 0.000000, 0.000485,    11.585,  83.57 %
Temps Démodulation : 0.000001, 0.000000, 0.000055,  1226.982,   0.79 %
Temps Décodage     : 0.000005, 0.000000, 0.000206,    47.833,   5.06 %
Temps Check        : 0.000001, 0.000000, 0.000095,   360.366,   0.67 %
================================================================================
```

Analyse des graphes :

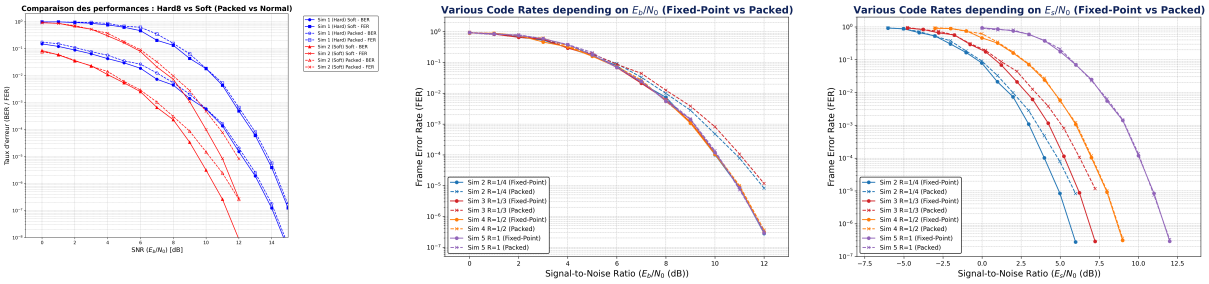


Fig. 5. – Résultats des simulations Packed Fixed-Point $K = 32$

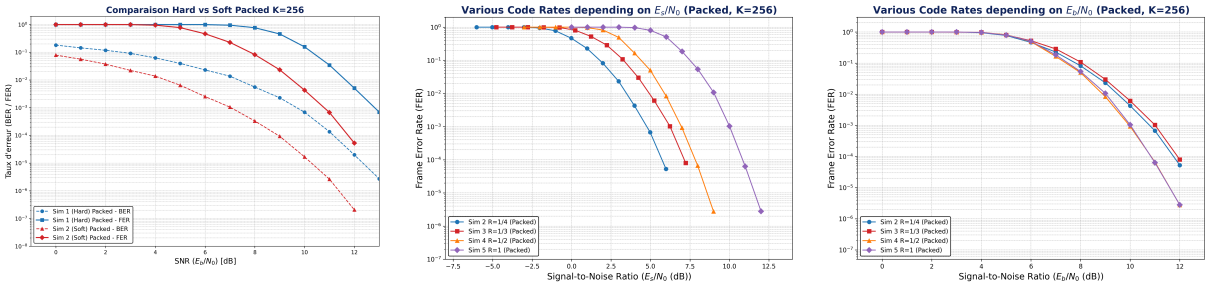


Fig. 6. – Résultats des simulations Packed Fixed-Point $K = 256$

Tout comme pour l'implémentation flottante, les décalages observés sur les simulations 2 et 3 en virgule fixe sont exclusivement liés aux paramètres matériels (S et F), tandis que les autres simulations garantissent la justesse du décodage.

La véritable puissance de l'empaquetage de bits combiné aux instructions vectorielles (virgule fixe) se révèle sur les statistiques de grandes trames ($K = 256$). La saturation efficace des registres *SIMD* permet une accélération massive des débits d'exécution : la vitesse de génération de la **Source** est multipliée par **3,5** (passant de 41 à 145 Mbps), et celle de l'**Encodeur à Répétition** par **4,5** (passant de 185 à 863 Mbps).

Axe I - Speeding up the overall simulation throughput

Task I-1. Multi-threading [difficulty: medium]

> *For this task you will use either pthread or the C++11 threads to parallelize the entire communication chain. The main idea is to have as many threads as cores and each core runs a standalone simulation (no synchronization between the communication chain blocks). The only thing that need to be synchronized is the frame errors counter. To reduce synchronization overhead, the frame counter can be synchronized loosely or using an atomic. Be careful, to make it work you need to carefully initialize the seeds in the generator and in the channel. If all the threads have the same seeds, the parallelization will be useless...*

Dans cette partie, nous avons décidé de créer un nouveau fichier pour gérer les threads indépendamment. Nous avons utilisé `pthread` (C) et avons rendu atomiques les variables `n_bit_errors`, `n_frame_errrors` et `n_frame` qui sont partagées et modifiées entre les différentes simulations. Chaque thread continue leur simulation jusqu'à que `n_frame_errors` atteint `f_max`, déclenchant l'arrêt de l'ensemble des threads. Nous avons utilisé les 6 cœurs disponible pour faire les simulations.

Les résultats :
Sur le graphique situé à gauche ($K = 32$) des différentes figures, la simulation 1 n'a été exécutée que jusqu'à un SNR de 14. La valeur pour le SNR 15 est donc omise. Néanmoins, une exécution complète produirait logiquement des résultats inscrits dans la même tendance que les simulations précédentes.

L'utilisation du flag d'optimisation **-O3** lors de la compilation ne nous permet pas d'obtenir des statistiques de performance (débit) représentatives pour la version classique (scalaire). L'analyse du débit se concentrera donc exclusivement sur la comparaison entre nos implémentations *SIMD* (NEON) et Packed. En revanche, les résultats de précision (BER et FER) restent totalement exploitables. Nous pouvons ainsi comparer les taux d'erreur des versions Packed et NEON avec la version de base pour valider l'exactitude des simulations. Si nécessaire, des tests ultérieurs compilés sans optimisation vectorielle pourront être menés pour quantifier précisément le gain de temps par rapport au code d'origine.

1. Thread Standard Vs Thread Packed

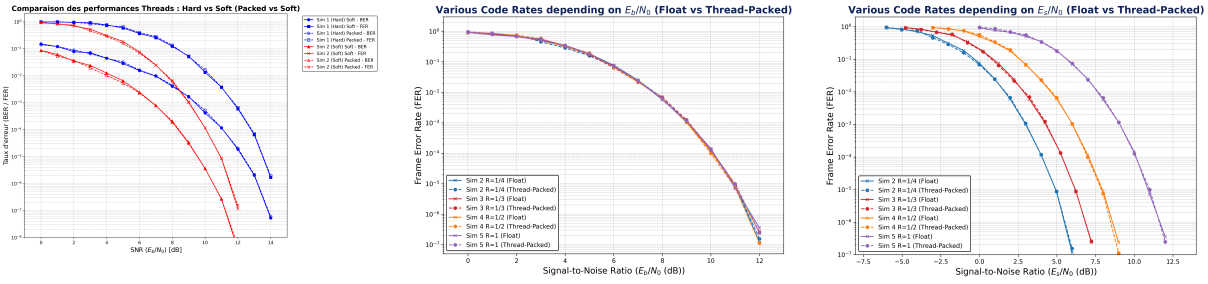


Fig. 7. – Résultats des simulations Thread Packed Flottant $K = 32$

Pour $K = 32$, l'implémentation *Thread Packed* est validée : les résultats obtenus sont presque identiques à ceux de la version de référence. L'optimisation *bit-packing* couplée au multi-threading est donc parfaitement fonctionnelle pour les algorithmes à décision *Hard* et *Soft*.

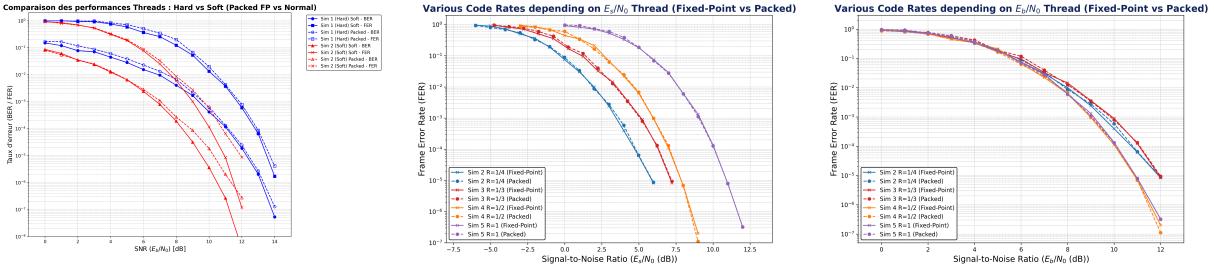


Fig. 8. – Résultats des simulations Thread Packed FP $K = 32$

Pour $K = 32$, les résultats de l'implémentation *Thread Packed FP* (virgule fixe) sont également concluants. L'approche *bit-packing* en multi-threading reste opérationnelle en décisions *Hard* et *Soft FP*, à condition de calibrer correctement les paramètres de quantification S et F.

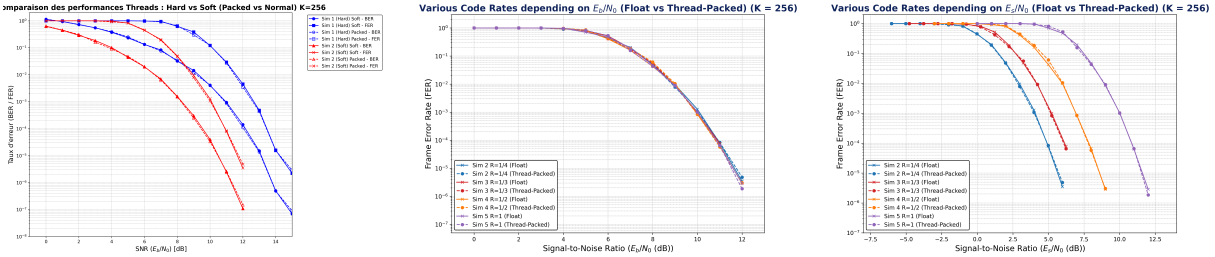


Fig. 9. – Résultats des simulations Thread Packed Flottant $K = 256$

Pour $K = 256$, les courbes se superposent parfaitement, confirmant l'exactitude des probabilités calculées. Le code est donc entièrement fonctionnel et validé lors d'une exécution multi-threadée répartie sur 6 cœurs.

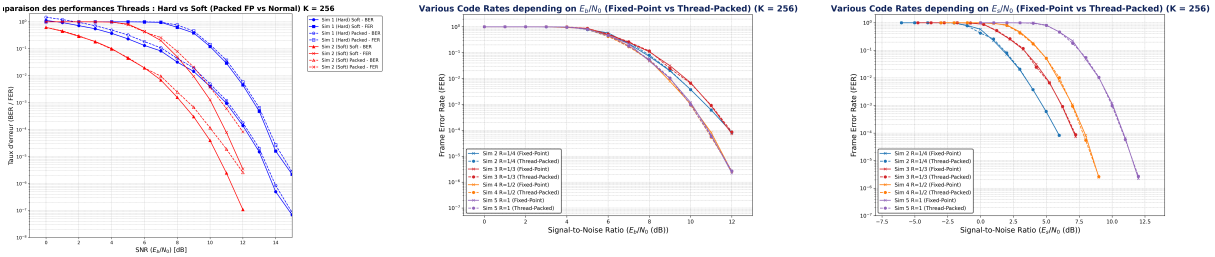


Fig. 10. – Résultats des simulations Thread Packed FP $K = 256$

On observe cependant la même limitation qu'auparavant concernant l'approche FP : pour les simulations 2 et 3, les valeurs actuelles de S et F induisent un écart (un peu mais pas beaucoup, au milieu. juste un écart) par rapport aux simulations de référence. Un ajustement fin de ces deux paramètres permettra d'obtenir les bonnes valeurs lors des prochaines exécutions.

Afin de quantifier l'apport du multi-threading, nous avons analysé les données de la simulation 5 à travers trois configurations distinctes :

Version de référence (séquentielle) :
Dans le fichier `/resultats_stats_normaux/sim5_stats.txt`, le débit observé est d'environ **2,35 Mbps**.

```
Eb/N0,Es/N0,sigma,be,fe,fn,BER,FER,total time,avg time,Mbps
...
12,12.00,0.18,100,100,333191478,0.00,0.00,4526.988379,0.000014,2.355236
```

Version Thread Standard ($K = 32$) :
Dans le fichier `resultats_csv_thread/K32/Standard/sim5_soft8.csv`, le débit grimpe à **12,11 Mbps**. L'utilisation des threads permet ici de multiplier le débit de la simulation de base par un facteur d'environ **5,14**.

```
Eb/N0,Es/N0,sigma,be,fe,fn,BER,FER,total time,avg time,Mbps
...
12,12.00,0.18,100,100,307979615,0.00,0.00,28627.206778,0.000093,12.109616
```

Version Thread Standard ($K = 256$) :
Dans le fichier `resultats_csv_thread/K256/Standard/sim5_soft8.csv`, le débit atteint **54,01 Mbps**. Ce résultat montre une accélération d'un facteur **4,46** par rapport à la version $K = 32$, et d'environ **22,9** par rapport à la simulation de référence.

```
Eb/N0,Es/N0,sigma,be,fe,fn,BER,FER,total time,avg time,Mbps
...
12,12.00,0.18,100,100,45095833,0.00,0.00,7685.044257,0.000170,54.014678
```

- **Version Thread Packed ($K = 256$) :**
  Enfin, l'analyse du fichier `resultats_csv_thread/K256/Packed/sim5_soft8.csv` révèle un débit maximal de 58,29 Mbps. Par rapport à l'implémentation *Packed* séquentielle, on observe ici un gain de performance d'un facteur 4.

  ```
  Eb/N0,Es/N0,sigma,be,fe,fn,BER,FER,total time,avg time,Mbps
  ...
  12,12.00,0.18,100,100,36965003,0.00,0.00,5900.469950,0.000160,58.286293
  ```

Synthèse de l'analyse :
Ces résultats mettent en évidence que le levier majeur d'accélération du débit réside fondamentalement dans l'exploitation du multi-threading. L'utilisation de threads « normaux » permet d'augmenter le débit global de manière drastique, d'autant plus lorsque la taille des données (K) augmente.

De plus, lorsque la machine cible dispose de plusieurs cœurs physiques, une approche d'optimisation purement séquentielle (traitement bloc par bloc au niveau de l'architecture) s'avère nettement moins optimisée qu'une parallélisation globale des tâches via les threads.

2. Thread Standard Vs Thread NEON

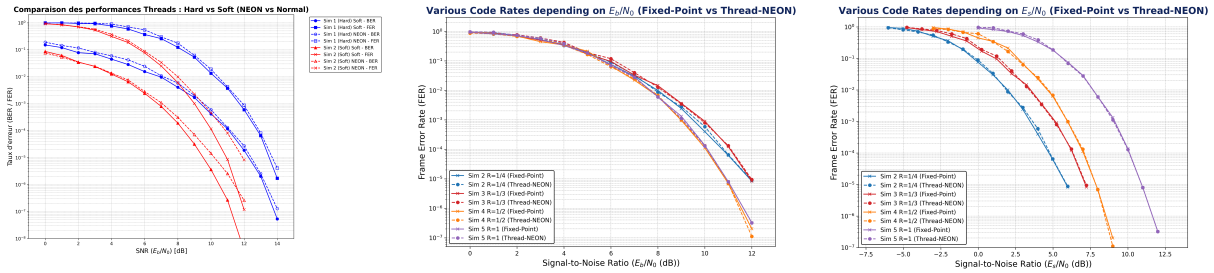


Fig. 11. – Résultats des simulations Thread NEON Flottant $K = 32$

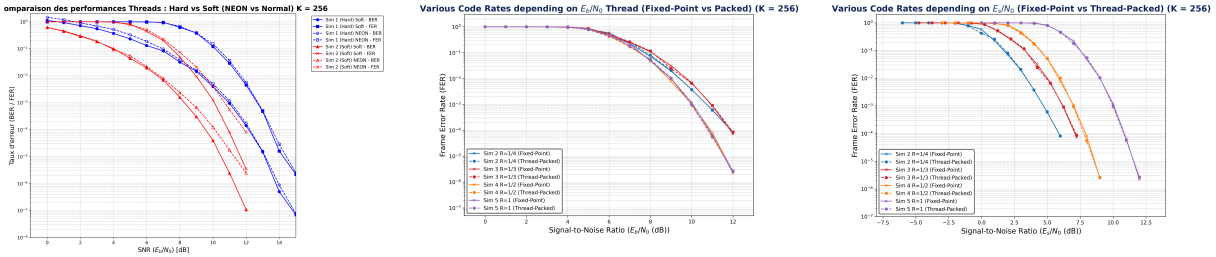


Fig. 12. – Résultats des simulations Thread NEON Flottant $K = 256$

Dans le fichier `resultats_csv_thread/K32/Standard/sim5_soft8.csv` :

```
Eb/N0,Es/N0,sigma,be,fe,fn,BER,FER,total time,avg time,Mbps
...
12,12.00,0.18,100,100,312623673,0.00,0.00,29097.075033,0.000093,12.128384
```

L'analyse de cette section s'appuie sur la simulation 5, très pertinente ici puisqu'elle valide le fonctionnement tout en démontrant un gain de performance majeur. Avec un débit mesuré à 12 Mbps (cf. données CSV ci-dessus), contre 2,395 Mbps pour l'implémentation précédente (**tâches II-1-2-3**), on observe que la vitesse de traitement a été pratiquement multipliée par cinq. Ainsi l'intégration des threads vient parfaire les optimisations précédentes. Alors que les instructions *SIMD* (NEON) accélèrent le traitement au niveau des données, le multi-threading permet de paralléliser l'exécution au niveau des tâches. En combinant ces deux approches, nous avons pu observer une augmentation drastique du débit global. Cette synergie exploite pleinement l'architecture du processeur et se traduit par une accélération majeure lors de la phase de test des trames.

3. Thread Packed Vs Thread Neon

Dans cette sous-section, nous confrontons les performances des implémentations **Thread Packed** et **Thread NEON** en analysant les statistiques d'exécution pour $K = 32$.

Statistiques d'exécution de la version NEON :

Dans le fichier `resultats_stats_thread/K32/Neon/sim1_hard8.txt` :

```
================================================================================
Bloc 14               Avg (s)    Min (s)    Max (s)    Avg Mbps   % Temps
--------------------------------------------------------------------------------
Temps Source       : 0.000007, 0.000000, 0.044539,     4.609,   5.66 %
Temps Répétition   : 0.000009, 0.000000, 0.023810,    14.756,   7.07 %
Temps Modulation   : 0.000006, 0.000000, 0.042442,    22.118,   4.72 %
Temps AWGN         : 0.000035, 0.000005, 0.088586,     3.686,  28.30 %
Temps Démodulation : 0.000006, 0.000000, 0.026676,    22.916,   4.55 %
Temps Décodage     : 0.000005, 0.000000, 0.023718,     5.872,   4.44 %
Temps Check        : 0.000005, 0.000000, 0.030818,     5.926,   4.40 %
================================================================================
```

Statistiques d'exécution de la version Packed :

Dans le fichier `resultats_stats_thread/K32/Packed/sim1_hard8.txt` :

```
================================================================================
Bloc 14               Avg (s)    Min (s)    Max (s)    Avg Mbps   % Temps
--------------------------------------------------------------------------------
Temps Source       : 0.000006, 0.000000, 0.067590,     5.430,   4.84 %
Temps Répétition   : 0.000006, 0.000000, 0.021533,    22.495,   4.67 %
Temps Modulation   : 0.000006, 0.000000, 0.023632,    22.878,   4.60 %
Temps AWGN         : 0.000035, 0.000006, 0.062534,     3.667,  28.67 %
Temps Démodulation : 0.000005, 0.000000, 0.022633,    23.460,   4.48 %
Temps Décodage     : 0.000009, 0.000000, 0.022839,     3.544,   7.42 %
Temps Check        : 0.000005, 0.000000, 0.003313,     5.913,   4.45 %
================================================================================
```

Comparaison des débits finaux :

- Fichier `resultats_csv_thread/K32/Neon/sim1_hard8.csv` : **9,41 Mbps**

```
Eb/N0,Es/N0,sigma,be,fe,fn,BER,FER,total time,avg time,Mbps
...
14,7.98,0.28,100,100,24394468,0.00,0.00,2993.376971,0.000123,9.414011
```

- Fichier `resultats_csv_thread/K32/Packed/sim1_hard8.csv` : **9,49 Mbps**

```
Eb/N0,Es/N0,sigma,be,fe,fn,BER,FER,total time,avg time,Mbps
...
14,7.98,0.28,100,100,24238099,0.00,0.00,2950.766438,0.000122,9.491080
```

Analyse et limites de la comparaison :
À première vue, les débits globaux sont extrêmement proches (9,41 Mbps contre 9,49 Mbps). Cependant, par un souci de transparence quant à notre méthodologie, il est important de signaler un biais de compilation lors de ces tests.

En effet, lors de l'exécution des simulations en *bit-packing*, nous avons conservé les flags d'optimisation standards du compilateur (comme `-O3`). Par conséquent, le compilateur s'est chargé d'optimiser de lui-même nos opérations de manipulation de bits en utilisant le jeu d'instructions NEON. Nous ne comparons donc pas une approche scalaire Packed pure contre une approche vectorielle NEON, mais plutôt du NEON implicite avec *bit packing* contre du NEON explicite.

Cela explique pourquoi les résultats globaux sont si semblables et fait que cet essai ne peut servir d'indicateur fiable pour mesurer l'efficacité du *bit-packing* isolé.

Néanmoins, l'analyse des sous-blocs révèle des différences d'architecture intéressantes malgré l'optimisation automatique : la tâche **Répétition** s'avère plus véloce avec l'approche Packed (22,49 Mbps contre 14,75 Mbps pour le code NEON explicite), tandis que le **Décodage** reste plus performant dans notre version NEON manuelle (5,87 Mbps contre 3,54 Mbps).

Afin d'isoler l'apport réel du *bit-packing* algorithmique dans de futurs travaux, il conviendra de recompiler cette configuration en forçant la désactivation de l'optimisation automatique.

Conclusion Générale

Au terme de ce projet, l'analyse globale de nos résultats démontre que l'approche multi-threadée constitue l'optimisation la plus performante parmi toutes les solutions explorées. C'est véritablement la synergie entre deux niveaux d'optimisation distincts qui permet d'atteindre des débits optimaux et d'exploiter la pleine puissance de l'architecture :

- **La vectorisation (SIMD / NEON)**, qui accélère le traitement au niveau des données (parallélisation spatiale).
- **Le multi-threading**, qui distribue l'exécution de ces tâches sur les différents cœurs physiques de la machine (parallélisation temporelle).

Cette union représente l'aboutissement et le point culminant de notre démarche d'accélération logicielle.

Cependant, l'analyse approfondie de nos métriques met en lumière une limite structurelle persistante : le bloc **AWGN** (ajout du bruit gaussien). L'analyse des profils d'exécution révèle que ce module constitue le principal facteur limitant de la chaîne, représentant à lui seul près de 30 % du temps de calcul total.

Étant donné que cette étape critique limite la vitesse globale du système, l'optimisation poussée des autres modules (modulation, décodage, source) ne produit plus qu'un gain marginal sur le temps total d'exécution. Par conséquent, toute amélioration significative des performances globales nécessitera inévitablement une refonte et une optimisation en profondeur du générateur AWGN.

Néanmoins, si ce constat est facile à dresser sur le papier, sa résolution technique est une toute autre affaire. La génération logicielle de variables aléatoires de haute qualité suivant une loi normale nécessite des algorithmes complexes.

Configuration et lancement

Procédure de compilation

- aller dans le répertoire build : `cd build`
- créer le makefile : `cmake ../src`
- compiler : `make all`

Lancer le code séquentiel

Depuis `mini_projet_MOHAMED_DAI/build`

./simulation -m ? -M ? -s ? -e 100 -K ? -N ? -D ? avec :

- m [min_SNR float] the first included Eb/N0 SNR to simulate (in dB),
- M [max_SNR float] the last included Eb/N0 SNR to simulate (in dB),
- s [step_val float] the constant step between two SNR points,
- e [f_max uint] the number of frame errors to reach to explore one SNR point,
- K [info_bits uint] the number of information bits,
- N [codeword_size uint] the codeword size (has to be a multiple of K otherwise the program should return an error),
- D [« rep-hard »|« rep-soft »|« rep-hard8 »|« rep-soft8 »|« rep-hard8-neon »|« rep-soft8-neon »|« rep-bit-packing-hard »|« rep-bit-packing-soft »|« rep-bit-packing-hard8 »|« rep-bit-packing-soft8 » string] select the decoder type.

Lancer le code avec les thread

Depuis `mini_projet_MOHAMED_DAI/build`

./simulation_thread -m ? -M ? -s ? -e 100 -K ? -N ? -D ? avec :

- m [min_SNR float] the first included Eb/N0 SNR to simulate (in dB),
- M [max_SNR float] the last included Eb/N0 SNR to simulate (in dB),
- s [step_val float] the constant step between two SNR points,
- e [f_max uint] the number of frame errors to reach to explore one SNR point,
- K [info_bits uint] the number of information bits,
- N [codeword_size uint] the codeword size (has to be a multiple of K otherwise the program should return an error),
- D [« rep-hard »|« rep-soft »|« rep-hard8 »|« rep-soft8 »|« rep-hard8-neon »|« rep-soft8-neon »|« rep-bit-packing-hard »|« rep-bit-packing-soft »|« rep-bit-packing-hard8 »|« rep-bit-packing-soft8 » string] select the decoder type.

Présentation des scripts d'analyse Python

Le répertoire `mini_projet_MOHAMED_DAI/resultat` contient un ensemble de scripts Python dédiés à la génération des graphiques. Cette sous-section détaille le rôle de chaque fichier selon sa nomenclature.

1. Comparaison des décisions Soft

Tous les scripts dont le nom commence par `graph` génèrent les courbes de FER en fonction de $\frac{E_s}{N_0}$ et $\frac{E_b}{N_0}$ pour les simulations 2 à 5.

Préfixe `graph_soft_` (Flottant standard) :

- **`normal_packed`** : Compare l'implémentation de référence et le *bit-packing* (Soft) pour $K = 32$.
- **`normal_packed_K256`** : Idem pour $K = 256$.

Préfixe `graph_soft8_` (Virgule fixe - FP) :

- **`normal_neon`** : Compare la version de référence et l'optimisation NEON (Soft FP) pour $K = 32$.
- **`normal_neon_K256`** : Idem pour $K = 256$.
- **`normal_packed`** : Compare la version de référence et le *bit-packing* (Soft FP) pour $K = 32$.
- **`normal_packed_K256`** : Idem pour $K = 256$.

Préfixe `graph_softthread_` (Flottant standard + Multi-threading 6 cœurs) :

- **`normal_packed`** : Compare la référence et le *bit-packing* en Soft (flottant standard), multi-threadé, pour $K = 32$.
- **`normal_packed_K256`** : Idem pour $K = 256$.

Préfixe `graph_soft8thread_` (Virgule fixe + Multi-threading 6 cœurs) :

- **`normal_neon`** : Compare la référence et NEON en Soft FP, multi-threadé, pour $K = 32$.
- **`normal_neon_K256`** : Idem pour $K = 256$.
- **`normal_packed`** : Compare la référence et le *bit-packing* en Soft FP, multi-threadé, pour $K = 32$.
- **`normal_packed_K256`** : Idem pour $K = 256$.

2. Comparaison des décisions Hard et Soft

Les scripts listés ci-dessous génèrent les courbes comparatives de BER et FER entre la simulation 1 (décision Hard) et la simulation 2 (décision Soft).

Préfixe `sim1_sim2_` :

- **`neon`** : Compare la référence et NEON (Hard vs Soft FP) pour $K = 32$.
- **`neon_k256`** : Idem pour $K = 256$.
- **`packed`** : Compare la référence et le *bit-packing* (Hard vs Soft flottant) pour $K = 32$.
- **`packed_k256`** : Idem pour $K = 256$.
- **`packed_FP`** : Compare la référence et le *bit-packing* (Hard vs Soft FP) pour $K = 32$.
- **`packed_FP_k256`** : Idem pour $K = 256$.
- **`threadneon`** : Compare la référence et NEON (Hard vs Soft FP), multi-threadé sur 6 cœurs, pour $K = 32$.
- **`threadneon_k256`** : Idem pour $K = 256$.
- **`threadpacked`** : Compare la référence et le *bit-packing* (Hard vs Soft flottant), multi-threadé, pour $K = 32$.
- **`threadpacked_k256`** : Idem pour $K = 256$.
- **`threadpackedFP`** : Compare la référence et le *bit-packing* (Hard vs Soft FP), multi-threadé, pour $K = 32$.
- **`threadpackedFP_k256`** : Idem pour $K = 256$.